



02

Data Manipulation With pandas



Adding a dataframe where columns are not identical

Case I : A subset

df1

	ID	Name	Major	GPA
0	6780001	Ann Molive	ICCH	3.2
1	6780002	Ben Kenobi	ICJD	NaN
2	6780003	Peter Cruiser	ICBA	3.3
3	6780004	Bob Hanger	ICFS	3.5
4	6780005	Moe Otto	ICCS	3.1

narrower_df

	ID	Name
0	6781111	Jacky Chow
1	6781112	Jet Loo

```
dfx = pd.concat([df1, narrower_df])  
dfx
```

	ID	Name	Major	GPA
0	6780001	Ann Molive	ICCH	3.2
1	6780002	Ben Kenobi	ICJD	NaN
2	6780003	Peter Cruiser	ICBA	3.3
3	6780004	Bob Hanger	ICFS	3.5
4	6780005	Moe Otto	ICCS	3.1
0	6781111	Jacky Chow	NaN	NaN
1	6781112	Jet Loo	NaN	NaN

Adding a dataframe where columns are not identical

Case II : totally different

df1

	ID	Name	Major	GPA
0	6780001	Ann Molive	ICCH	3.2
1	6780002	Ben Kenobi	ICJD	NaN
2	6780003	Peter Cruiser	ICBA	3.3
3	6780004	Bob Hanger	ICFS	3.5
4	6780005	Moe Otto	ICCS	3.1

```
dfy = pd.concat([df1, another_df])  
dfy
```

	ID	Name	Major	GPA	Hobby	Club
0	6780001.0	Ann Molive	ICCH	3.2	NaN	NaN
1	6780002.0	Ben Kenobi	ICJD	NaN	NaN	NaN
2	6780003.0	Peter Cruiser	ICBA	3.3	NaN	NaN
3	6780004.0	Bob Hanger	ICFS	3.5	NaN	NaN
4	6780005.0	Moe Otto	ICCS	3.1	NaN	NaN
0	NaN	NaN	NaN	NaN	Reading	Board Game
1	NaN	NaN	NaN	NaN	Running	Debate

	Hobby	Club
0	Reading	Board Game
1	Running	Debate

NaN created by the operation

Adding a dataframe where columns are not identical

Case III : some columns are the same

df1

	ID	Name	Major	GPA
0	6780001	Ann Molive	ICCH	3.2
1	6780002	Ben Kenobi	ICJD	NaN
2	6780003	Peter Cruiser	ICBA	3.3
3	6780004	Bob Hanger	ICFS	3.5
4	6780005	Moe Otto	ICCS	3.1

```
dfz = pd.concat([df1, foo_df])  
dfz
```

	ID	Name	Major	GPA	Weight	Height
0	6780001	Ann Molive	ICCH	3.2	NaN	NaN
1	6780002	Ben Kenobi	ICJD	NaN	NaN	NaN
2	6780003	Peter Cruiser	ICBA	3.3	NaN	NaN
3	6780004	Bob Hanger	ICFS	3.5	NaN	NaN
4	6780005	Moe Otto	ICCS	3.1	NaN	NaN
0	6780101	NaN	ICCH	NaN	65.0	170.0
1	6780102	NaN	ICCS	NaN	59.0	180.0

	ID	Major	Weight	Height
0	6780101	ICCH	65	170
1	6780102	ICCS	59	180

NaN created by the operation

Inner Join

- The inner keyword will look at the structures of both dataframes. If any columns are available on both, those columns will be in the result.
- Hence, this is structural.
- It doesn't look at the value inside the cells.

df1.loc[0, 'Major'] = NaN

```
dfz = pd.concat([df1, foo_df], join="inner")
```

dfz

ID	Major
0	6780001
1	6780002
2	6780003
3	6780004
4	6780005
0	6780101
1	6780102

foo_df

ID	Major	Weight	Height
0	6780101	ICCH	65
1	6780102	ICCS	59

Columns that doesn't survive into result

Columns that appear on both dataframes

Outer Join

- the default value of the `concat` function

df1

ID	Name	Major	GPA
0	6780001	Ann Molive	NaN
1	6780002	Ben Kenobi	ICJD
2	6780003	Peter Cruiser	ICBA
3	6780004	Bob Hanger	ICFS
4	6780005	Moe Otto	ICCS

foo_df

ID	Major	Weight	Height
0	6780101	ICCH	65
1	6780102	ICCS	59

```
dfz = pd.concat([df1, foo_df], join="outer")
```

dfz

ID	Name	Major	GPA	Weight	Height
0	6780001	Ann Molive	NaN	3.2	NaN
1	6780002	Ben Kenobi	ICJD	NaN	NaN
2	6780003	Peter Cruiser	ICBA	3.3	NaN
3	6780004	Bob Hanger	ICFS	3.5	NaN
4	6780005	Moe Otto	ICCS	3.1	NaN
0	6780101	NaN	ICCH	65.0	170.0
1	6780102	NaN	ICCS	59.0	180.0

NaN from operation



Adding Columns

Adding along the Direction of Column

col_concat = pd.concat([df1, df2, df3], axis=1)

col_concat

	A	B	C	D	A	B	C	D	A	B	C	D
0	A0	B0	C0	D0	A4	B4	C4	D4	A8	B8	C8	D8
1	A1	B1	C1	D1	A5	B5	C5	D5	A9	B9	C9	D9
2	A2	B2	C2	D2	A6	B6	C6	D6	A10	B10	C10	D10
3	A3	B3	C3	D3	A7	B7	C7	D7	A11	B11	C11	D11

df1

	A	B	C	D
0	A0	B0	C0	D0
1	A1	B1	C1	D1
2	A2	B2	C2	D2
3	A3	B3	C3	D3

df2

	A	B	C	D
0	A4	B4	C4	D4
1	A5	B5	C5	D5
2	A6	B6	C6	D6
3	A7	B7	C7	D7

df3

	A	B	C	D
0	A8	B8	C8	D8
1	A9	B9	C9	D9
2	A10	B10	C10	D10
3	A11	B11	C11	D11

Problem: Is this what you need?

```
col_concat = pd.concat([df1, df2, df3], axis=1)
col_concat
```

	A	B	C	D	A	B	C	D	A	B	C	D
0	A0	B0	C0	D0	A4	B4	C4	D4	A8	B8	C8	D8
1	A1	B1	C1	D1	A5	B5	C5	D5	A9	B9	C9	D9
2	A2	B2	C2	D2	A6	B6	C6	D6	A10	B10	C10	D10
3	A3	B3	C3	D3	A7	B7	C7	D7	A11	B11	C11	D11

```
col_concat['A']
```

	A	A	A
0	A0	A4	A8
1	A1	A5	A9
2	A2	A6	A10
3	A3	A7	A11

Columns that didn't make it to the result.

Adding Columns with Different Indexes

df1

	A	B	C	D
0	A0	B0	C0	D0
1	A1	B1	C1	D1
2	A2	B2	C2	D2
3	A3	B3	C3	D3

df2

	D	E	F	G
0	A4	B4	C4	D4
1	A5	B5	C5	D5
2	A6	B6	C6	D6
3	A7	B7	C7	D7

df3

	A	D	G	H
0	A8	B8	C8	D8
1	A9	B9	C9	D9
2	A10	B10	C10	D10
3	A11	B11	C11	D11

row_concat = pd.concat([df1, df2, df3])
row_concat

	A	B	C	D	E	F	G	H
0	A0	B0	C0	D0	NaN	NaN	NaN	NaN
1	A1	B1	C1	D1	NaN	NaN	NaN	NaN
2	A2	B2	C2	D2	NaN	NaN	NaN	NaN
3	A3	B3	C3	D3	NaN	NaN	NaN	NaN
4	NaN	NaN	NaN	NaN	A4	B4	C4	D4
5	NaN	NaN	NaN	NaN	A5	B5	C5	D5
6	NaN	NaN	NaN	NaN	A6	B6	C6	D6
7	NaN	NaN	NaN	NaN	A7	B7	C7	D7
8	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
9	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
10	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
11	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
12	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
13	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
14	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
15	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
16	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
17	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
18	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
19	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
20	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
21	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
22	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
23	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
24	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
25	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
26	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
27	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
28	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
29	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
30	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
31	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
32	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
33	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
34	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
35	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
36	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
37	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
38	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
39	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
40	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
41	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
42	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
43	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
44	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
45	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
46	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
47	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
48	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
49	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
50	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
51	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
52	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
53	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
54	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
55	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
56	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
57	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
58	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
59	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
60	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
61	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
62	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
63	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
64	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
65	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
66	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
67	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
68	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
69	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
70	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
71	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
72	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
73	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
74	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
75	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
76	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
77	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
78	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
79	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
80	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
81	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
82	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
83	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
84	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
85	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
86	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
87	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
88	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
89	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
90	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
91	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
92	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
93	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
94	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
95	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
96	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
97	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
98	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
99	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN

NaN that exist in the result, but not in the source dataframe

Adding Columns with Different Indexes

```
row_concat = pd.concat([df1, df2, df3])
row_concat
```

	A	B	C	D	E	F	G	H
0	A0	B0	C0	D0	NaN	NaN	NaN	NaN
1	A1	B1	C1	D1	NaN	NaN	NaN	NaN
2	A2	B2	C2	D2	NaN	NaN	NaN	NaN
3	A3	B3	C3	D3	NaN	NaN	NaN	NaN
4	NaN	NaN	NaN	A4	B4	C4	D4	NaN
5	NaN	NaN	NaN	A5	B5	C5	D5	NaN
6	NaN	NaN	NaN	A6	B6	C6	D6	NaN
7	NaN	NaN	NaN	A7	B7	C7	D7	NaN
8	A8	NaN	NaN	B8	NaN	NaN	C8	D8
9	B9	NaN	NaN	B9	NaN	NaN	C9	D9
10	A10	NaN	NaN	B10	NaN	NaN	C10	D10
11	A11	NaN	NaN	B11	NaN	NaN	C11	D11

Concatenate Columns With Different Rows

df1

	A	B	C	D
0	A0	B0	C0	D0
1	B1	C1	D1	
2	A2	B2	C2	D2
3	A3	B3	C3	D3

df2

	D	E	F	G
2	A4	B4	C4	D4
3	A5	B5	C5	D5
4	A6	B6	C6	D6
5	A7	B7	C7	D7

df3

	A	D	G	H
2	A8	B8	C8	D8
3	A9	B9	C9	D9
6	A10	B10	C10	D10
7	A11	B11	C11	D11

col_concat = pd.concat([df1, df2, df3], axis=1, join="inner")
col_concat

	A	B	C	D	D	E	F	G	A	D	G	H
2	A2	B2	C2	D2	A4	B4	C4	D4	A8	B8	C8	D8
3	A3	B3	C3	D3	A5	B5	C5	D5	A9	B9	C9	D9

Inner Join

Rows that doesn't exist on the result

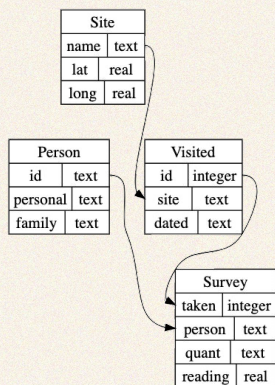
Duplicate column name, as we concatenate by row label



Merging

Combining dataframes based on values

A Survey Database



person

	ident	personal	family
0	dyer	William	Dyer
1	pb	Frank	Pabodie
2	lake	Anderson	Lake
3	roe	Valentina	Roerich
4	danforth	Frank	Danforth

visited

	ident	site	dated
0	619	DR-1	1927-02-08
1	622	DR-1	1927-02-10
2	734	DR-3	1939-01-07
3	735	DR-3	1939-01-12
4	751	DR-3	1939-02-26
5	752	DR-3	NaN
6	837	MSK-4	1932-01-14
7	844	DR-1	1932-03-22

site

	name	lat	long
0	DR-1	-49.85	-128.57
1	DR-3	-47.15	-126.72
2	MSK-4	-48.87	-123.40

survey

	taken	person	quant	reading
0	619	dyer	rad	9.82
1	619	dyer	sal	0.13
2	622	dyer	rad	7.80
3	622	dyer	sal	0.09
4	734	pb	rad	8.41
5	734	lake	sal	0.05
6	734	pb	temp	-21.50
7	735	pb	rad	7.22
8	735	NaN	sal	0.06
9	735	NaN	temp	-26.00
10	751	pb	rad	4.35
11	751	pb	temp	-18.50
12	751	lake	sal	0.10
13	752	lake	rad	2.19
14	752	lake	sal	0.09
15	752	lake	temp	-16.00
16	752	roe	sal	41.60
17	837	lake	rad	1.46
18	837	lake	sal	0.21
19	837	roe	sal	22.50
20	844	roe	rad	11.25

<https://training.galaxyproject.org/training-material/topics/data-science/tutorials/sql-advanced/tutorial.html>

One-to-One Merge

There exist a One-to-One match for 'name' of site dataframe, and 'site' of visited_subset dataframe, so pandas just append the right dataframe's row to each corresponding row of the left dataframe.

site

	name	lat	long
0	DR-1	-49.85	-128.57
1	DR-3	-47.15	-126.72
2	MSK-4	-48.87	-123.40

visited_subset

	ident	site	dated
0	619	DR-1	1927-02-08
2	734	DR-3	1939-01-07
6	837	MSK-4	1932-01-14

one2one_merge = site.merge(visited_subset, left_on="name", right_on="site")
one2one_merge

	name	lat	long	ident	site	dated
0	DR-1	-49.85	-128.57	619	DR-1	1927-02-08
1	DR-3	-47.15	-126.72	734	DR-3	1939-01-07
2	MSK-4	-48.87	-123.40	837	MSK-4	1932-01-14

one2one_join = site.join(visited_subset)
one2one_join

	name	lat	long	ident	site	dated
0	DR-1	-49.85	-128.57	619.0	DR-1	1927-02-08
1	DR-3	-47.15	-126.72	NaN	NaN	NaN
2	MSK-4	-48.87	-123.40	734.0	DR-3	1939-01-07

No index match, so pandas fill in those entries with NaN

Many-to-One Merge

```
visited.loc[2, 'site'] = NaN
visited
Many2One_merge = visited.merge(site, left_on="site", right_on='name')
Many2One_merge
```

ident	site	dated	name	lat	long
0	619	DR-1	1927-02-08	DR-1	-49.85 -128.57
1	622	DR-1	1927-02-10	DR-3	-47.15 -126.72
2	734	DR-3	1930-01-07	MSK-4	-48.87 -123.40
3	735	DR-3	1930-01-12	DR-3	-47.15 -126.72
4	751	DR-3	1930-02-26	DR-3	-47.15 -126.72
5	752	DR-3	NaN	DR-3	-47.15 -126.72
6	837	MSK-4	1932-01-14	MSK-4	-48.87 -123.40
7	844	DR-1	1932-03-22	DR-1	-49.85 -128.57

Data that didn't survive into the result, as there's no match to NaN of visited dataframe, to anything in 'name' of site dataframe

One-to-Many Merge

```
visited.loc[2, 'site'] = NaN
visited
One2Many_merge = site.merge(visited, left_on="name", right_on='site')
One2Many_merge
```

name	lat	long	ident	site	dated
DR-1	-49.85	-128.57	619	DR-1	1927-02-08
DR-3	-47.15	-126.72	622	DR-1	1927-02-10
MSK-4	-48.87	-123.40	734	DR-3	1930-01-07
DR-3	-47.15	-126.72	735	DR-3	1930-01-12
DR-3	-47.15	-126.72	751	DR-3	1930-02-26
DR-3	-47.15	-126.72	752	DR-3	NaN
MSK-4	-48.87	-123.40	837	MSK-4	1932-01-14
DR-1	-49.85	-128.57	844	DR-1	1932-03-22

Data that didn't survive into the result, as there's no match to NaN of visited dataframe, to anything in 'name' of site dataframe

Many-to-Many Merge 1/2

```
Many2Many_merge = enrollment_df.merge(exam_df, left_on="Course", right_on="Course")
Many2Many_merge
```

ID	Course	Exam
0	1001	ICCS161 Midterm
1	1001	ICCS161 Final
2	1002	ICCS161 Midterm
3	1002	ICCS161 Final

```
Many2Many_merge_2 = enrollment_df_2.merge(exam_df, left_on="Course", right_on="Course")
Many2Many_merge_2
```

ID	Course	Exam
0	1001	ICCS161 Midterm
1	1001	ICCS161 Final
2	1002	ICCS161 Midterm
3	1002	ICCS161 Final

Data that didn't survive into the result, as there's no match to NaN of visited dataframe, to anything in 'name' of site dataframe

Many-to-Many Merge 2/2

```
enrollment_df_2.set_index('Course', inplace=True)
enrollment_df_2
```

ID	Course
0	ICCS161
1	ICCS161
2	ICCS331

```
exam_df.set_index('Course', inplace=True)
exam_df
```

Exam	Course
Midterm	ICCS161
Final	ICCS161

```
Many2Many_merge_3 = enrollment_df_2.join(exam_df, how="outer")
Many2Many_merge_3
```

ID	Exam	Course
0	Midterm	ICCS161
1	Final	ICCS161
2	Midterm	ICCS331
3	Final	ICCS331

Same as Many-to-Many merge 2/1. The ICCS331 row survived the operation and got NaN for the column label it doesn't have.

```
Many2Many_merge_3 = enrollment_df_2.join(exam_df, how="inner")
Many2Many_merge_3
```

ID	Exam	Course
0	Midterm	ICCS161
1	Final	ICCS161

Same as Many-to-Many merge 2/1. The ICCS331 row did not survive the operation